

SINCRONIZACIÓN DE PROCESOS

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN

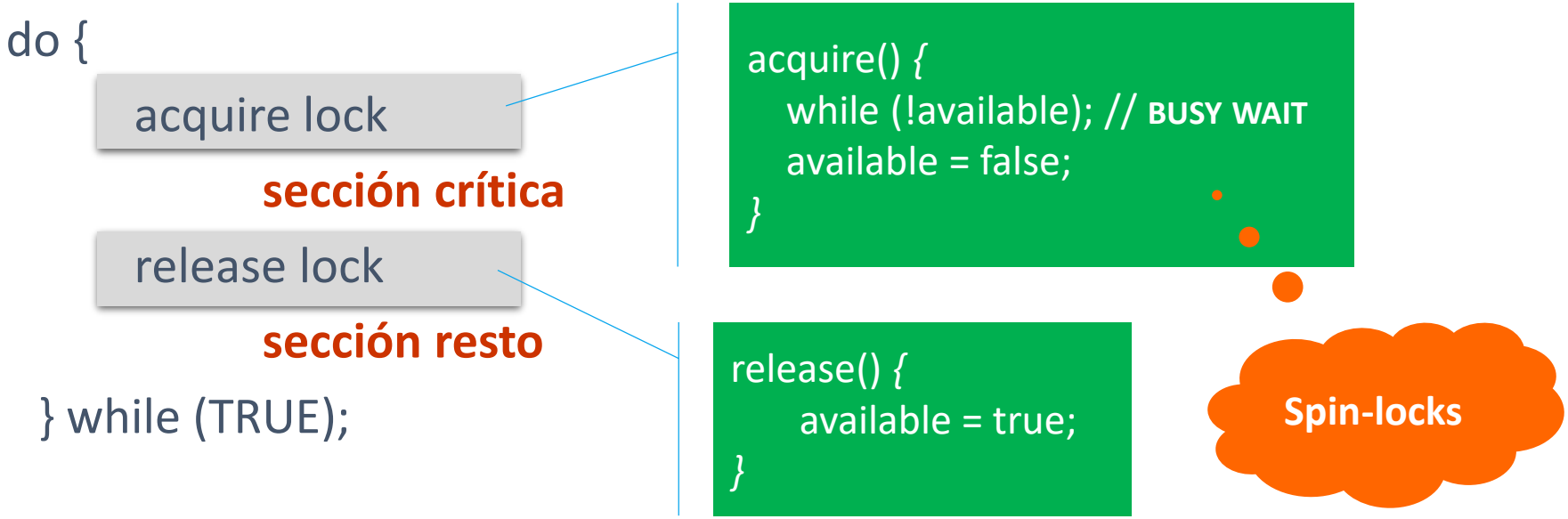
UNIVERSIDAD NACIONAL DEL SUR

KMC © 2026

Agenda

1. Fundamentos
2. El Problema de la Sección Crítica
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. Soluciones por Hardware
5. **Soluciones por Software**
    1. **Locks**
    2. **Semáforos**
    3. **Monitores**
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos

Exclusión Mutua usando locks



mutex locks – se utilizan específicamente para la exclusión mutual

Semáforos

Semáforo

> Es una herramienta de sincronización.

- Semáforo ***S*** se define como una variable entera
- Dos operaciones standard modifican el valor de un semáforo ***S***
  wait() y **signal()**
- Puede ser accedido solo por dos operaciones indivisibles (*deben ser atómicas*):

wait (***S***)

```
while S≤ 0 do no-op;
S := S – 1;
```

signal (***S***)

```
S := S + 1;
```

Semáforo - Herramienta General de Sincronización

- Semáforo de **Cuenta (Contador)** – el valor entero puede tener un rango sobre un dominio sin restricciones.
- Semáforo **Binario** – el valor entero puede tener un rango solo entre 0 y 1.
- Un semáforo puede utilizarse para alcanzar exclusión mutua

```
Semáforo S;    //  inicializado en 1
wait (S);
        Sección Crítica
 signal (S);
```

Semáforo - Herramienta General de Sincronización

Para sincronización

- Ejecute *B* en P_j solo después que *A* ejecute en P_i
- Use el semáforo *flag* inicializado a 0
- Código:

P_i	P_j
⋮	⋮
A	***wait(flag)***
signal(flag)	*B*

Implementación del Semáforo

Condición

Debe garantizar que dos procesos no puedan ejecutar **wait()** y **signal()** sobre el mismo semáforo al mismo tiempo.



Implementación del Semáforo

- La implementación se convierte en el problema de la sección crítica donde el **código del wait** y **el signal** son la sección crítica.
  - Podemos tener ahora espera ocupada en la implementación de la sección crítica porque:
    - El código de implementación es corto
    - Poca espera ocupada si la sección crítica está raramente invocada

- Note que las aplicaciones pueden pasar y gastar mucho tiempo en secciones críticas, entonces no es una buena solución utilizar semáforos implementados con espera ocupada.

Implementación de Semáforo sin Espera Ocupada

- Con cada semáforo hay asociada una cola de espera.
  Cada entrada en dicha cola tiene dos datos:
  - valor (de tipo entero)
  - puntero al próximo registro en la lista

- Dos operaciones:
  - **block** – ubica el proceso invocando la operación en la apropiada cola de espera.
  - **wakeup** – remueve uno de los procesos en la cola de espera y lo ubica en la cola de listos.

Implementación de Semáforo sin Espera Ocupada

Las operaciones del semáforo se definen como

***wait*(*S*):**

```
S.value := S.value – 1;
if S.value < 0
   then begin
          agregue este proceso a S.L;
          block;
   end;
```

***signal*(*S*):**

```
S.value := S.value + 1;
if S.value ≤ 0
   then begin
          remueva un proceso P de S.L;
          wakeup(P);
    end;
```

Implementación de *S* con Semáforos Binarios

- Estructuras de datos:

  var *S1: binary-semaphore; S2: binary-semaphore;*
  S3: binary-semaphore; C: integer;

- Inicialización:

  S1 = S3 = 1; *S2* = 0; *C* = valor inicial del semáforo *S*

operación ***wait***

```
wait(S3);
wait(S1);
C := C – 1;
if C < 0
then begin
        signal(S1);
        wait(S2);
end
else signal(S1);
signal(S3);
```

operación ***signal***

```
wait(S1);
C := C + 1;
if C ≤ 0 then signal(S2);
signal(S1);
```

Cuidado con el mal uso de los semáforos

Interbloqueo e Inanición

- **INTERBLOQUEO** – dos o más procesos están esperando indefinidamente por un evento que puede ser causado por solo uno de los procesos que esperan.
- Sean S y Q dos semáforos inicializados a 1

P0	P1
wait(S);	wait(Q);
wait(Q);	wait(S);
⋮	⋮
signal(S);	signal(Q);
signal(Q);	signal(S);

- **INANICIÓN** – bloqueo indefinido. Un proceso no puede ser removido nunca de la cola del semáforo en el que fue suspendido.
- **INVERSIÓN DE PRIORIDADES**

Problema

Realizar la sincronización de la siguiente secuencia de ejecución. Cada proceso está asociado con una letra diferente.

ABCABCABCABCABCABCABC…….

```
semaphore semA = 1
semaphore semB = 0
semaphore semC = 0
```

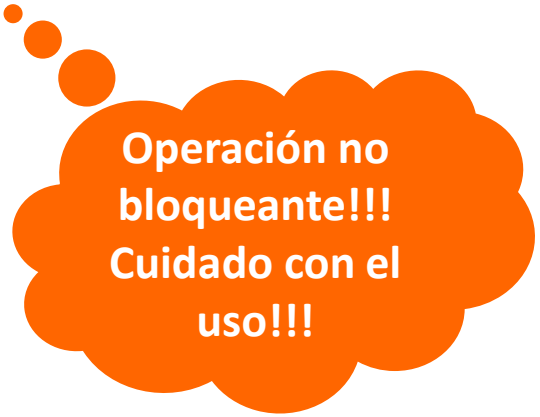
La solución está asociada con la inicialización

Proceso A	Proceso B	Proceso C
repeat	repeat	repeat
wait(semA)	wait(semB)	wait(semC)
Tarea A	Tarea B	Tarea C
signal(semB)	signal(semC)	signal(semA)
until false	until false	until false

Semáforos

- Incorrecto uso de las operaciones sobre semáforos:
  - signal (mutex) …. wait (mutex)
  - wait (mutex) … wait (mutex)
  - Omitir el wait (mutex) o el signal (mutex) (o ambos)

- Extensión
  - Incorporación de la operación ***wait-try*** sobre un semáforo.



Monitores

Es un constructor de sincronización de alto nivel que permite compartir en forma segura un tipo de dato abstracto entre procesos concurrentes.

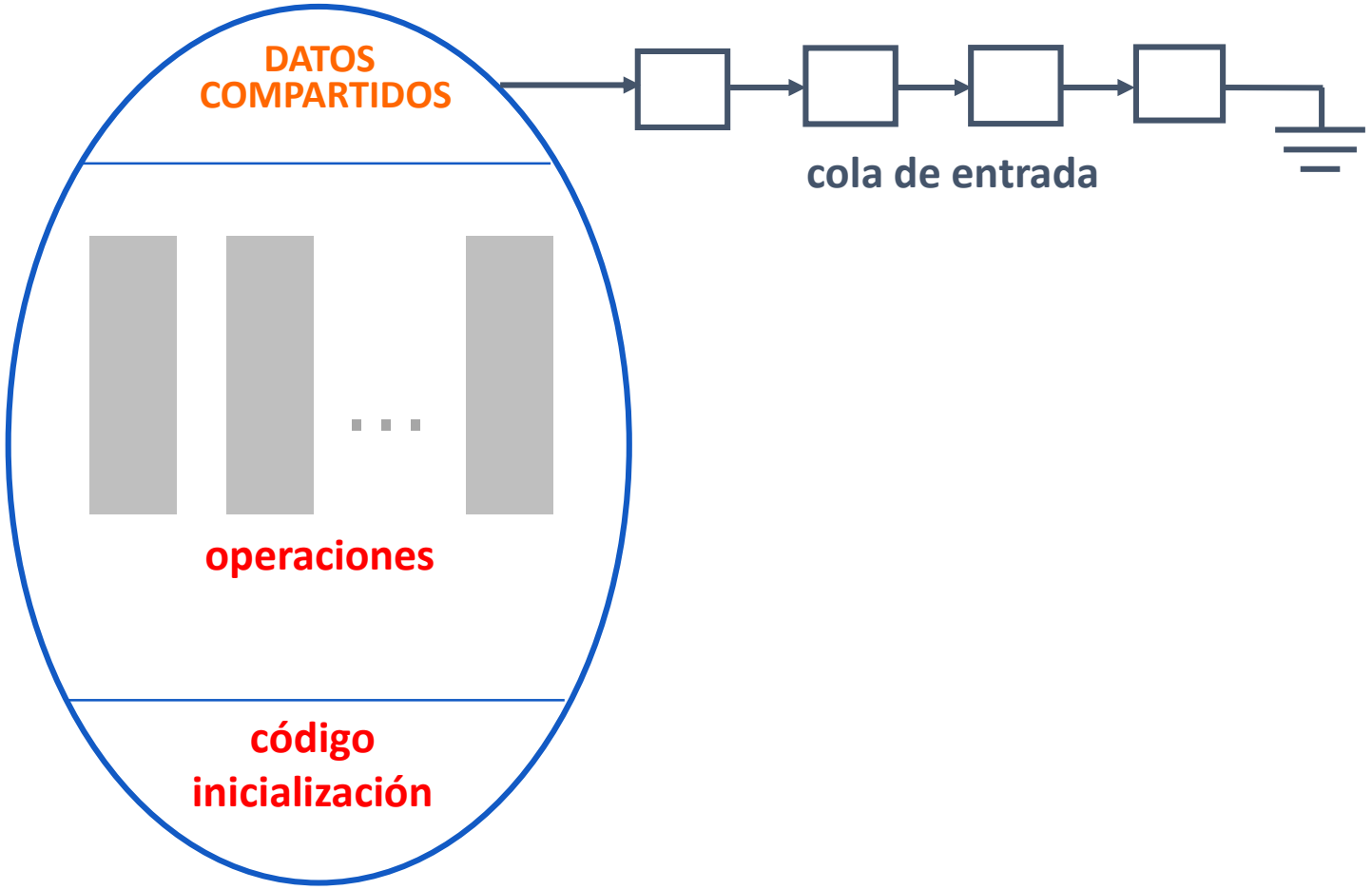
```
monitor monitor-nombre
{
  // declaración de variables compartidas

  procedure P1 (…) { …. }

  procedure Pn (…) {……}

   código de inicialización(…) { … }
}
```

Vista esquemática de un Monitor



Variables de Condición

Las variables de condición son un mecanismo que se utiliza para la sincronización.

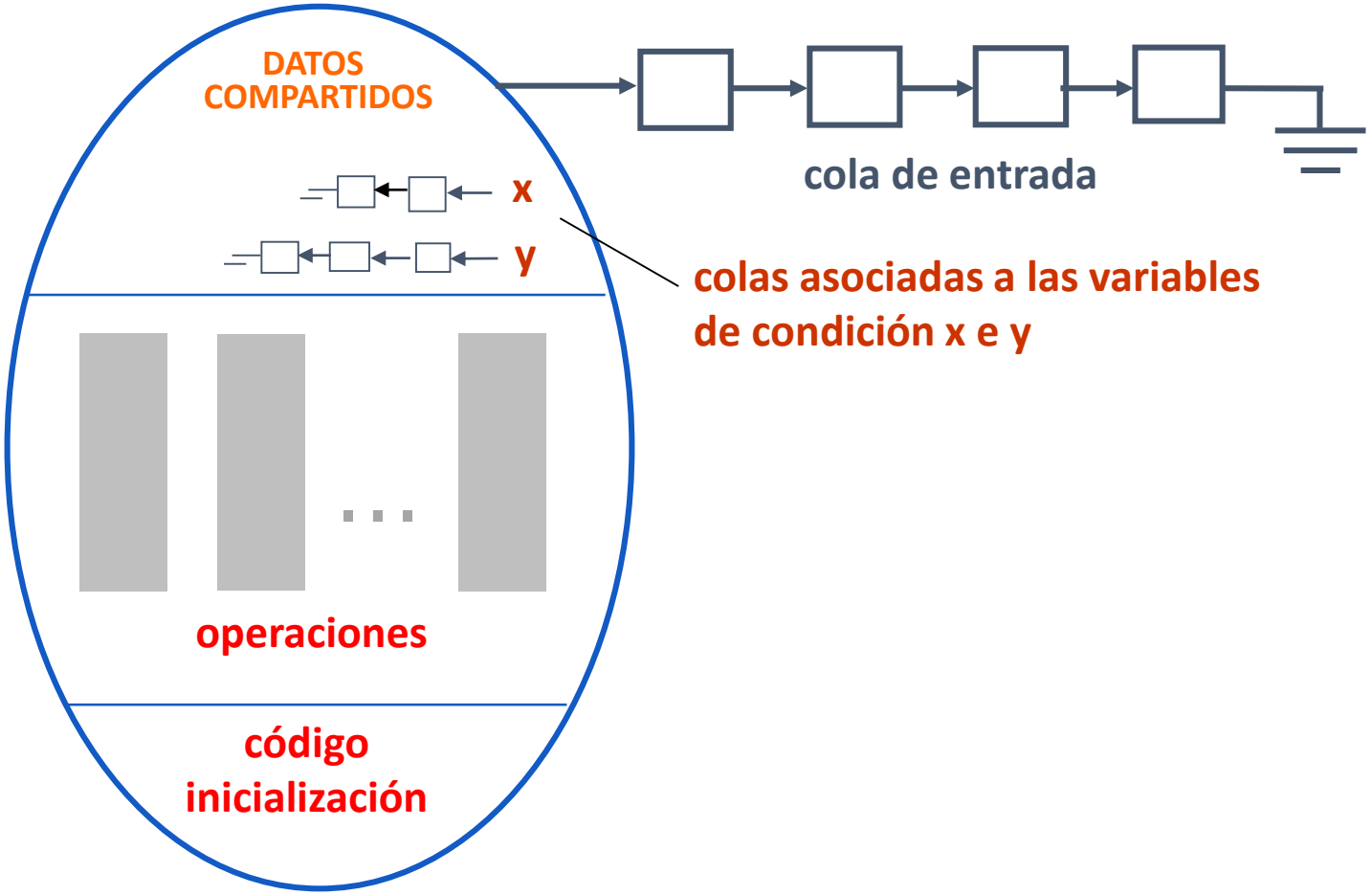
Por ejemplo para definir dos variables de condición

Condition x, y

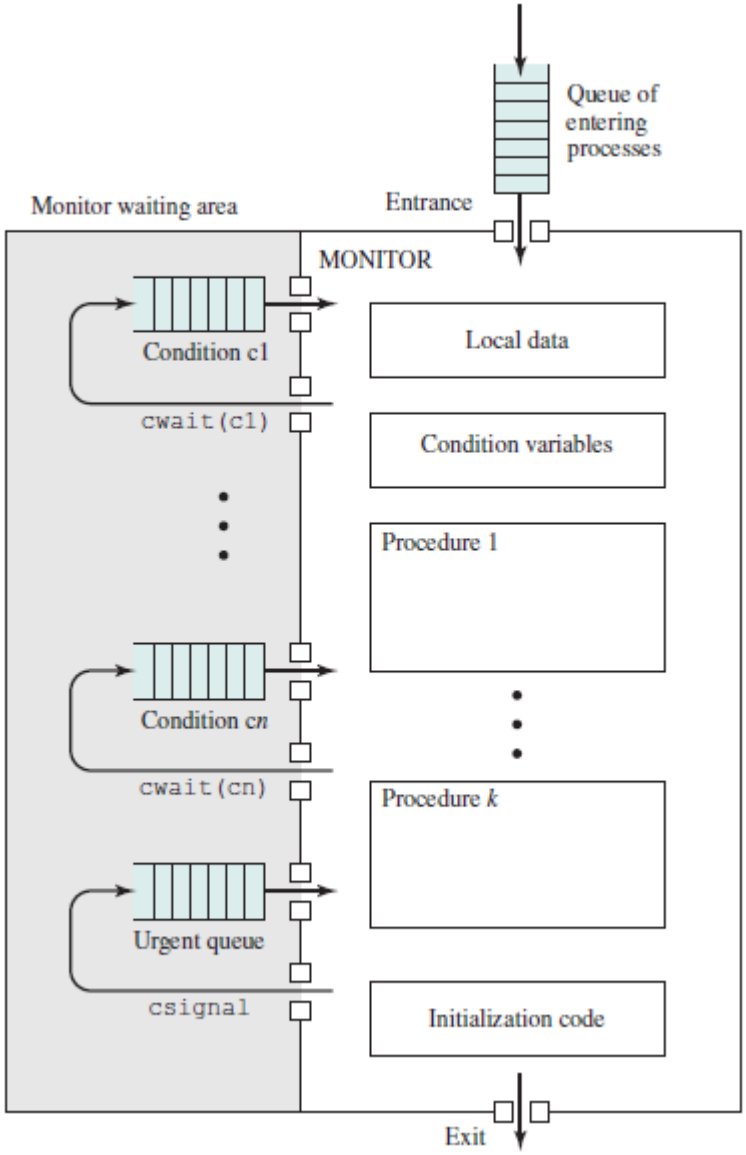
Dos operaciones sobre una variable de condición

- **x.wait ()** – el proceso que invoca esa operación es suspendido
- **x.signal ()** – reinicia uno de los procesos (si hay alguno) que invocó **x.wait ()**

Monitor con Variables de Condición



Monitor con Variables de Condición



Agenda

1. Fundamentos
2. El Problema de la Sección Crítica
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. Soluciones por Hardware
5. Soluciones por Software
    1. Locks
    2. Semáforos
    3. Monitores
6. **Problemas Clásicos**
7. Ejemplos de Sincronización en los Sistemas Operativos

Problemas Clásicos de Sincronización

- Problema del Buffer Limitado
- Problema de Lectores y Escritores
- Problema de los Filósofos Cenando

Problema del Buffer Limitado

Datos compartidos

type *item* = …

var *buffer* = …

lleno, vacío : semáforo de conteo;

mutex: semáforo binario; (o como mutex-lock)

nextp, nextc: item;

Inicialización Semáforos

lleno :=0; *vacío* := *n*; *mutex* :=1;

Problema del Buffer Limitado

Proceso Productor	Proceso Consumidor
repeat … produce un ítem en *nextp* … *wait*(*vacío*); ***wait*(*mutex*);** … agregue *nextp* al buffer … ***signal*(*mutex*);** *signal*(*lleno*); **until** *false*;	**repeat** *wait*(*lleno*) ***wait*(*mutex*);** … remueve un ítem de *buffer* a *nextc* … ***signal*(*mutex*);** *signal*(*vacío*); … consume el ítem en *nextc* … **until** *false*;

Problema del Buffer Limitado

```
monitor ProdCons
        condition lleno, vacío;
        integer contador;

procedure insertar(ítem: integer)
begin
      if contador == N then lleno.wait();
      Insertar_ítem(ítem);
      contador := contador + 1;
      If contador == 1 then vacío.signal()
end;

function remover: integer
begin
      If contador == 0 then vacío.wait();
      Remover = remover_ítem;
      contador := contador - 1;
      if contador == N-1 then lleno.signal()
end;

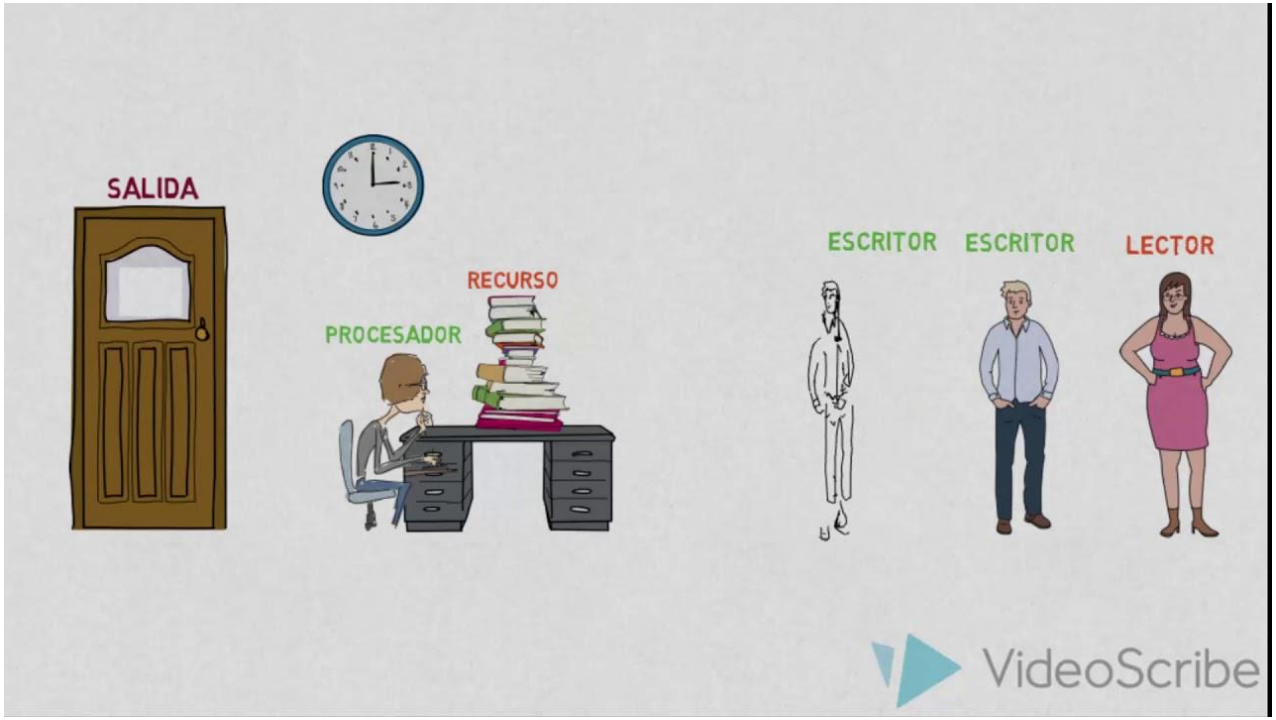
        contador := 0;
end monitor;
```

Problema del Buffer Limitado

```
procedure productor
begin
    while true do
    begin
     ítem = produce_ítem;
     ProdCons.insertar(ítem);
    end
end;
```

```
procedure consumidor
begin
    while true do
    begin
        ítem = ProdCons.remover;
        Consume_ítem(ítem);
    end
end;
```

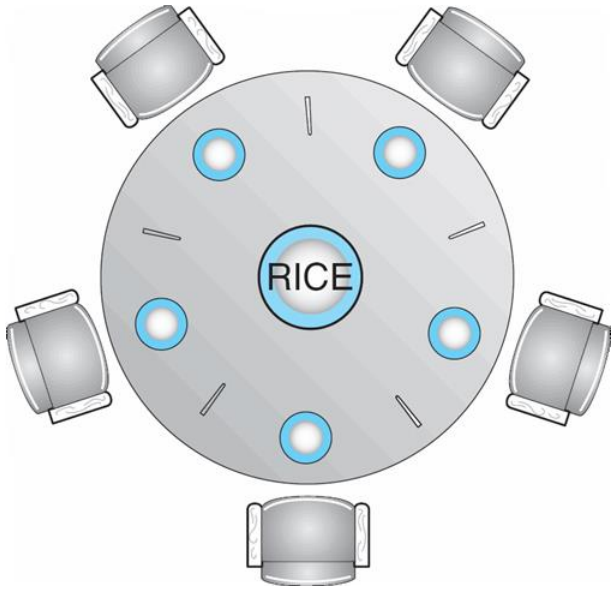
Problema Lectores y Escritores



https://youtu.be/Ck-zAxDWNHg

Problema de los Filósofos Cenando

- Los filósofos pasan sus vidas pensando y comiendo.
- No interactúan con sus vecinos, cuando quieren comer requieren los 2 palillos, tomando de a uno a la vez para comer del tazón.
- Necesita ambos palillos para comer, cuando finalizan de comer los liberan.
- Hay una fuente de arroz y en el caso de 5 filósofos, hay 5 palillos.



Bibliografía:

- Silberschatz, A., Gagne G., y Galvin, P.B.; "Operating System Concepts", 7ma Edición 2009, 9na Edición 2012, 10ma Edición 2018.
- Stallings, W. "Operating Systems: Internals and Design Principles", Prentice Hall, 7ma Edición 2011, 8va Edición 2014, 9na Edición 2018.
- Tanenbaum, A.; "Modern Operating Systems", Pearson, 3rd Edición 2008, 4th Edición 2014, 5th. Edition 2022.